

Functions in C

C - How to Program, 9th edition

Umer Shahid
Department of Electrical Engineering,
UET Lahore

October 15, 2025

What You'll Learn This Week

- How to break large programs into smaller, manageable pieces
- How to create and use functions
- Understanding function prototypes
- How information passes between functions
- Working with the C standard library
- Random number generation
- Building a complete game (Craps)
- Recursion basics

Why Do We Need Functions?

Problem: Large programs are difficult to:

- Write
- Understand
- Debug (find errors)
- Maintain (update later)

Solution: Divide and Conquer!

- Break program into smaller pieces
- Each piece solves a specific task
- Easier to manage and understand

Real-Life Analogy

Building a House:

- Don't do everything yourself
- Hire specialists:
 - Electrician
 - Plumber
 - Carpenter
- Each expert does their job
- You coordinate (like `main`)

Writing a Program:

- Don't write all code in `main`
- Create specialized functions:
 - `calculate()`
 - `display()`
 - `validate()`
- Each function has a job
- `main` coordinates

What is a Function?

Definition: A function is a block of code that:

1. Has a name
2. Performs a specific task
3. Can be called (used) multiple times
4. Can receive input (parameters)
5. Can return output (return value)

You've already used functions!

- `printf()` - displays output
- `scanf()` - reads input
- `main()` - starting point of program

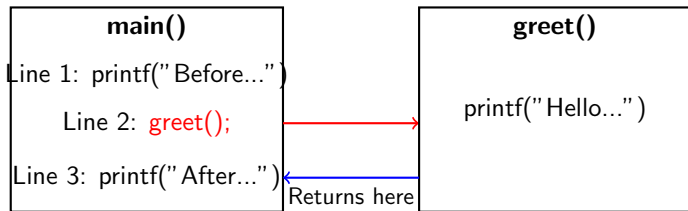
Your First Custom Function

```
1 #include <stdio.h>
2
3 // Function to display a greeting
4 void greet(void) {
5     printf("Hello from the function!\n");
6 }
7
8 int main(void) {
9     printf("Before calling function\n");
10    greet(); // Calling the function
11    printf("After calling function\n");
12    return 0;
13 }
```

Output:

Before calling function
Hello from the function!
After calling function

Understanding the Function Call



Execution Flow:

1. Start in `main`
2. When `greet()` is called, jump to that function
3. Execute function code
4. Return to `main` and continue

Parts of a Function

A function has several parts:

1. **Return Type** - What type of value it gives back
2. **Function Name** - What you call it
3. **Parameters** - Input values (optional)
4. **Function Body** - The code that runs
5. **Return Statement** - Sends value back (optional)

Let's look at each part in detail...

Return Type

Return Type tells what kind of value the function gives back:

```
1 int    sum(int a, int b)    // Returns an integer
2 double average(int x, int y) // Returns a decimal
3 char   getGrade(int marks)  // Returns a character
4 void   display(void)        // Returns nothing
```

Important:

- void means the function returns nothing
- If return type is int, you **must** return an integer
- Return type must match what you actually return

Function Name

Rules for naming functions:

- Start with a letter or underscore
- Can contain letters, digits, underscores
- Cannot use C keywords (like `int`, `if`, `return`)
- Case-sensitive (`Sum` $\neq$ `sum`)

Good Practice:

```
1 // Good - describes what function does
2 int calculateSum(int a, int b)
3 void displayMenu(void)
4 double findAverage(int x, int y)
5
6 // Bad - unclear names
7 int func1(int a, int b)
8 void xyz(void)
9 double f(int x, int y)
```

Parameters (Input)

Parameters are values you pass to the function:

```
1 // No parameters
2 void sayHello(void) {
3     printf("Hello!\n");
4 }
5
6 // One parameter
7 void printNumber(int num) {
8     printf("The number is: %d\n", num);
9 }
10
11 // Multiple parameters
12 int add(int x, int y) {
13     return x + y;
14 }
```

Note: Use void if no parameters needed

Return Statement

Return statement sends a value back to the caller:

```
1 int square(int num) {  
2     int result = num * num;  
3     return result; // Send result back  
4 }  
5  
6 int main(void) {  
7     int value = square(5); // value gets 25  
8     printf("Square: %d\n", value);  
9     return 0;  
10 }
```

Important:

- Only one value can be returned
- Return type must match function's declared type
- Execution stops when return is encountered

Example: Square Function

```
1 #include <stdio.h>
2
3 // Function prototype (declaration)
4 int square(int number);
5
6 int main(void) {
7     // Loop and calculate squares
8     for (int x = 1; x <= 10; ++x) {
9         printf("%d ", square(x));
10    }
11    puts("");
12    return 0;
13 }
14
15 // Function definition
16 int square(int number) {
17     return number * number;
18 }
```

Output: 1 4 9 16 25 36 49 64 81 100

Breaking Down the Example

Three parts of using a function:

1. **Function Prototype** (line 4)
 - Tells compiler the function exists
 - Written before `main`
 - Ends with semicolon
2. **Function Call** (line 10)
 - Actually uses the function
 - Passes value (argument) to function
 - Receives return value
3. **Function Definition** (lines 16-18)
 - Contains the actual code
 - Usually written after `main`
 - No semicolon after header

Function Prototype Explained

Why do we need prototypes?

```
1 int square(int number); // Prototype
```

- Tells compiler: "This function exists somewhere"
- Allows calling function before defining it
- Compiler checks if you're using it correctly
- Helps catch errors early

Parts of prototype:

- Return type: `int`
- Function name: `square`
- Parameter type: `int`
- Semicolon at end

Note: Parameter name is optional in prototype

`int square(int);` is also valid

Using Math Library

C provides built-in math functions in `<math.h>`:

```
1 #include <stdio.h>
2 #include <math.h>    // Must include for math functions
3
4 int main(void) {
5     printf("Square root of 900: %.2f\n", sqrt(900.0));
6     printf("2 raised to power 3: %.2f\n", pow(2, 3));
7     printf("Absolute value of -5: %.2f\n", fabs(-5.0));
8     return 0;
9 }
```

Output:

```
Square root of 900: 30.00
2 raised to power 3: 8.00
Absolute value of -5: 5.00
```

Common Math Functions

Function	Description	Example
<code>sqrt(x)</code>	Square root	<code>sqrt(25.0) = 5.0</code>
<code>pow(x, y)</code>	x raised to power y	<code>pow(2, 3) = 8.0</code>
<code>fabs(x)</code>	Absolute value	<code>fabs(-4.5) = 4.5</code>
<code>ceil(x)</code>	Round up	<code>ceil(4.2) = 5.0</code>
<code>floor(x)</code>	Round down	<code>floor(4.8) = 4.0</code>
<code>sin(x)</code>	Sine (x in radians)	<code>sin(0) = 0.0</code>
<code>cos(x)</code>	Cosine (x in radians)	<code>cos(0) = 1.0</code>
<code>log(x)</code>	Natural logarithm	<code>log(2.718) $\approx$ 1.0</code>

Important: All return double type

Pass by Value

In C, all arguments are passed "by value"

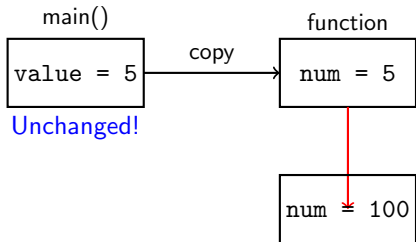
```
1 #include <stdio.h>
2
3 void tryToChange(int num) {
4     num = 100; // Changes local copy only
5     printf("Inside function: %d\n", num);
6 }
7
8 int main(void) {
9     int value = 5;
10    printf("Before: %d\n", value);
11    tryToChange(value);
12    printf("After: %d\n", value); // Still 5!
13    return 0;
14 }
```

Output:

Before: 5

Inside function: 100

Understanding Pass by Value



Key Point: Function receives a **copy** of the value
Changes to the copy don't affect the original

The Function Call Stack

What is a stack?

- Think of a stack of plates
- Add plate on top (PUSH)
- Remove plate from top (POP)
- Last In, First Out (LIFO)

Function calls use a stack:

- When function is called: push info on stack
- Info includes: return address, local variables
- When function returns: pop info from stack
- Previous function continues where it left off

Stack Example

Stack changes:

1. OS calls main

main: a=10

2. main calls square

square: x=10

main: a=10

3. square returns

main: a=10, result=100

4. main ends

(Stack empty)

```
1 int square(int x) {  
2     return x * x;  
3 }  
4  
5 int main(void) {  
6     int a = 10;  
7     int result;  
8     result = square(a);  
9     printf("%d", result);  
10 }
```

Generating Random Numbers

Use rand() function from <stdlib.h>:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void) {
5     // Generate 5 random numbers
6     for (int i = 1; i <= 5; i++) {
7         int randomNum = rand();
8         printf("%d\n", randomNum);
9     }
10    return 0;
11 }
```

Output: (example)

41
18467
6334
26500

Random Numbers in a Range

Problem: `rand()` gives very large numbers

Solution: Use modulo operator to limit range

```
1 // Random number from 1 to 6 (dice roll)
2 int die = 1 + (rand() % 6);
3
4 // Random number from 0 to 9
5 int digit = rand() % 10;
6
7 // Random number from 1 to 100
8 int percent = 1 + (rand() % 100);
```

Formula for range [a, b]:

$$\text{number} = a + (\text{rand()} \% (b - a + 1));$$

Problem: Same Numbers Every Time!

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void) {
5     for (int i = 1; i <= 5; i++) {
6         printf("%d ", 1 + (rand() % 6));
7     }
8     return 0;
9 }
```

Run 1: 6 6 5 5 6

Run 2: 6 6 5 5 6 (Same!)

Run 3: 6 6 5 5 6 (Same again!)

Why? rand() generates the same sequence each time

Solution: Seeding the Random Generator

Use `srand()` with current time:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h> // For time()
4
5 int main(void) {
6     srand(time(NULL)); // Seed with current time
7
8     for (int i = 1; i <= 5; i++) {
9         printf("%d ", 1 + (rand() % 6));
10    }
11    return 0;
12 }
```

Run 1: 3 1 4 2 6

Run 2: 5 5 1 3 2 (Different!)

Run 3: 2 4 6 1 5 (Different again!)

Best Practice: Call `srand(time(NULL))` once at start

Local vs Global Variables

Local Variables:

- Declared inside a function
- Exist only while function runs
- Destroyed when function ends
- Cannot be accessed from other functions

Global Variables:

- Declared outside all functions
- Exist for entire program
- Can be accessed by any function
- Generally should be avoided!

Local Variable Example

```
1 #include <stdio.h>
2
3 void function1(void) {
4     int x = 5;    // Local to function1
5     printf("In function1: x = %d\n", x);
6 }
7
8 void function2(void) {
9     int x = 10;   // Different x, local to function2
10    printf("In function2: x = %d\n", x);
11 }
12
13 int main(void) {
14     function1();
15     function2();
16     return 0;
17 }
```

Output:

In function1: x = 5

Global Variable Example

```
1 #include <stdio.h>
2
3 int globalVar = 100;  // Global variable
4
5 void function1(void) {
6     printf("Function1: globalVar = %d\n", globalVar);
7     globalVar = 200;  // Modifies global
8 }
9
10 void function2(void) {
11     printf("Function2: globalVar = %d\n", globalVar);
12 }
13
14 int main(void) {
15     printf("Main: globalVar = %d\n", globalVar);
16     function1();
17     function2();
18     return 0;
19 }
```

Static Local Variables

Static variables retain their value between function calls:

```
1 #include <stdio.h>
2
3 void counter(void) {
4     static int count = 0; // Initialized only once
5     count++;
6     printf("Count: %d\n", count);
7 }
8
9 int main(void) {
10     counter(); // Count: 1
11     counter(); // Count: 2
12     counter(); // Count: 3
13     return 0;
14 }
```

Key Point: static variables:

- Keep their value between calls
- Initialized only once

What is Recursion?

Recursion: When a function calls itself

Real-life example:

- Looking up a word in dictionary
- Definition uses another word you don't know
- Look up that word (recursion!)
- Eventually find words you know (base case)

Two essential parts:

1. **Base case:** Stopping condition
2. **Recursive case:** Function calls itself with simpler problem

Simple Recursion Example: Countdown

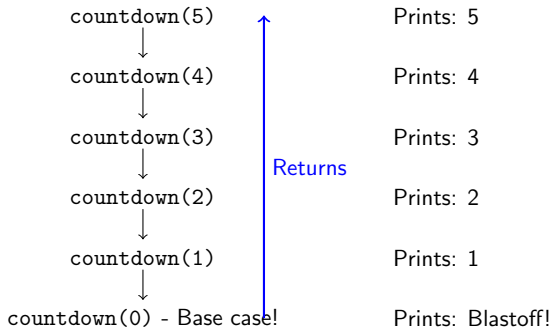
```
1 #include <stdio.h>
2
3 void countdown(int n) {
4     if (n == 0) {                // Base case
5         printf("Blastoff!\n");
6         return;
7     }
8     printf("%d\n", n);
9     countdown(n - 1);           // Recursive call
10 }
11
12 int main(void) {
13     countdown(5);
14     return 0;
15 }
```

Output:

5

4

How Countdown Works



Factorial Using Recursion

Mathematical definition:

- $n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$
- $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
- $0! = 1$ (by definition)

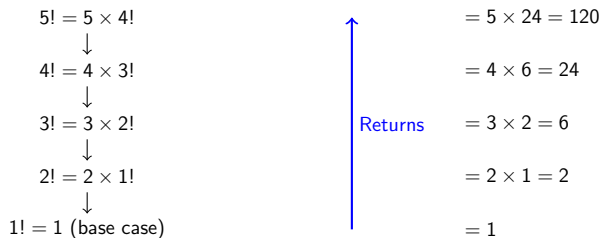
Recursive definition:

- $n! = n \times (n - 1)!$
- $5! = 5 \times 4!$

```
1 unsigned long long factorial(int n) {  
2     if (n <= 1) {                // Base case  
3         return 1;  
4     }  
5     else {                        // Recursive case  
6         return n * factorial(n - 1);  
7     }  
8 }
```

Factorial Visualization

Computing 5!:



Danger: Infinite Recursion!

What if there's no base case?

```
1 void infiniteRecursion(int n) {  
2     printf("%d\n", n);  
3     infiniteRecursion(n - 1);    // Never stops!  
4 }
```

Result:

- Function keeps calling itself
- Stack overflow error
- Program crashes

Always remember:

1. Have a base case
2. Make sure you reach the base case
3. Each recursive call should be simpler

Problem 1: Function to Find Maximum

Write a function to find maximum of two numbers

```
1 #include <stdio.h>
2 // Function prototype
3 int findMax(int a, int b);
4 int main(void) {
5     int num1 = 25, num2 = 40;
6     int maximum = findMax(num1, num2);
7     printf("Maximum is: %d\n", maximum);
8     return 0;
9 }
10 // Function definition
11 int findMax(int a, int b) {
12     if (a > b) {
13         return a;
14     }
15     else {
16         return b;
17     }
18 }
```

Problem 2: Function to Check Even/Odd

Write a function that returns 1 if number is even, 0 if odd

```
1 #include <stdio.h>
2
3 int isEven(int num);
4
5 int main(void) {
6     int number;
7     printf("Enter a number: ");
8     scanf("%d", &number);
9
10    if (isEven(number)) {
11        printf("%d is even\n", number);
12    }
13    else {
14        printf("%d is odd\n", number);
15    }
16    return 0;
17 }
```

Problem 3: Temperature Converter

Convert Celsius to Fahrenheit: $F = (C \times 9/5) + 32$

```
1 #include <stdio.h>
2
3 double celsiusToFahrenheit(double celsius);
4
5 int main(void) {
6     double temp_c = 25.0;
7     double temp_f = celsiusToFahrenheit(temp_c);
8     printf("%.1f C = %.1f F\n", temp_c, temp_f);
9     return 0;
10 }
11
12 double celsiusToFahrenheit(double celsius) {
13     return (celsius * 9.0 / 5.0) + 32.0;
14 }
```

Output: 25.0 C = 77.0 F

Note: Use double for decimal calculations

Problem 4: Function with Multiple Parameters

Calculate area of rectangle

```
1 #include <stdio.h>
2
3 int calculateArea(int length, int width);
4
5 int main(void) {
6     int l = 10, w = 5;
7     printf("Area of rectangle (%d x %d): %d\n",
8           l, w, calculateArea(l, w));
9     return 0;
10 }
11
12 int calculateArea(int length, int width) {
13     return length * width;
14 }
```

Output: Area of rectangle (10 x 5): 50

Key Points:

- Multiple parameters separated by commas

Problem 5: Power Function

Calculate $\text{base}^{\text{exponent}}$ using loops

```
1 #include <stdio.h>
2
3 int power(int base, int exp);
4
5 int main(void) {
6     printf("2^3 = %d\n", power(2, 3));
7     printf("5^2 = %d\n", power(5, 2));
8     return 0;
9 }
10
11 int power(int base, int exp) {
12     int result = 1;
13     for (int i = 0; i < exp; i++) {
14         result *= base;
15     }
16     return result;
17 }
```

Output:

Problem 6: Sum of Digits

Find sum of digits in a number

```
1 #include <stdio.h>
2
3 int sumOfDigits(int num);
4
5 int main(void) {
6     int number = 1234;
7     printf("Sum of digits in %d = %d\n",
8           number, sumOfDigits(number));
9     return 0;
10 }
11
12 int sumOfDigits(int num) {
13     int sum = 0;
14     while (num > 0) {
15         sum += num % 10;    // Add last digit
16         num /= 10;         // Remove last digit
17     }
18     return sum;
19 }
```

Problem 7: Prime Number Checker

```
1 #include <stdio.h>
2
3 int isPrime(int num);
4
5 int main(void) {
6     int n = 17;
7     if (isPrime(n)) {
8         printf("%d is prime\n", n);
9     } else {
10        printf("%d is not prime\n", n);
11    }
12    return 0;
13 }
14
15 int isPrime(int num) {
16     if (num <= 1) return 0;
17     for (int i = 2; i * i <= num; i++) {
18         if (num % i == 0) {
19             return 0; // Not prime
20         }
21     }
```

Problem 8: Recursive Sum

Sum of numbers from 1 to n using recursion

```
1 #include <stdio.h>
2
3 int sum(int n);
4
5 int main(void) {
6     printf("Sum of 1 to 5: %d\n", sum(5));
7     printf("Sum of 1 to 10: %d\n", sum(10));
8     return 0;
9 }
10
11 int sum(int n) {
12     if (n == 1) {           // Base case
13         return 1;
14     }
15     return n + sum(n - 1); // Recursive case
16 }
```

Output:

Problem 9: Print Pattern with Functions

```
1 #include <stdio.h>
2
3 void printStars(int count);
4 void printPattern(int rows);
5
6 int main(void) {
7     printPattern(5);
8     return 0;
9 }
10
11 void printStars(int count) {
12     for (int i = 0; i < count; i++) {
13         printf("* ");
14     }
15     printf("\n");
16 }
17
18 void printPattern(int rows) {
19     for (int i = 1; i <= rows; i++) {
20         printStars(i);
```

Problem 10: Menu-Driven Calculator

```
1 #include <stdio.h>
2
3 int add(int a, int b) { return a + b; }
4 int subtract(int a, int b) { return a - b; }
5 int multiply(int a, int b) { return a * b; }
6
7 int main(void) {
8     int choice, num1, num2;
9     printf("1.Add 2.Subtract 3.Multiply\n");
10    printf("Enter choice: ");
11    scanf("%d", &choice);
12    printf("Enter two numbers: ");
13    scanf("%d %d", &num1, &num2);
14
15    if (choice == 1)
16        printf("Result: %d\n", add(num1, num2));
17    else if (choice == 2)
18        printf("Result: %d\n", subtract(num1, num2));
19    else if (choice == 3)
20        printf("Result: %d\n", multiply(num1, num2));
```

Mistake 1: Forgetting Semicolon in Prototype

Wrong:

```
1 int square(int x)    // Missing semicolon!
2
3 int main(void) {
4     // ...
5 }
```

Correct:

```
1 int square(int x);   // Semicolon required
2
3 int main(void) {
4     // ...
5 }
```

Remember: Prototype is a statement, needs semicolon!

Mistake 2: Type Mismatch

Wrong:

```
1 int  getValue(void);
2
3 int  main(void) {
4     double x = getValue(); // OK but may lose data
5     return 0;
6 }
7
8 double getValue(void) { // Returns double but declared int!
9     return 3.14;
10 }
```

Correct:

```
1 double getValue(void); // Match the return type!
2
3 int  main(void) {
4     double x = getValue();
5     return 0;
6 }
```

Mistake 3: Not Returning a Value

Wrong:

```
1 int add(int a, int b) {  
2     int sum = a + b;  
3     // Forgot to return!  
4 }
```

Correct:

```
1 int add(int a, int b) {  
2     int sum = a + b;  
3     return sum; // Must return int value  
4 }  
5  
6 // Or more concisely:  
7 int add(int a, int b) {  
8     return a + b;  
9 }
```

Rule: If function has non-void return type, it **MUST** return a value!

Mistake 4: Wrong Number of Arguments

Wrong:

```
1 int add(int a, int b) {  
2     return a + b;  
3 }  
4  
5 int main(void) {  
6     int result = add(5); // Missing second argument!  
7     return 0;  
8 }
```

Correct:

```
1 int add(int a, int b) {  
2     return a + b;  
3 }  
4  
5 int main(void) {  
6     int result = add(5, 3); // Both arguments provided  
7     return 0;  
8 }
```

Mistake 5: Modifying Parameters Expecting Change

Wrong Assumption:

```
1 void increment(int x) {  
2     x = x + 1;    // Changes local copy only!  
3 }  
4  
5 int main(void) {  
6     int num = 5;  
7     increment(num);  
8     printf("%d\n", num);    // Still 5, not 6!  
9     return 0;  
10 }
```

Remember: Pass by value means changes don't affect original

Solution: Return the new value

```
1 int increment(int x) {  
2     return x + 1;  
3 }  
4  
5 int main(void) {
```

Function Design Principles

1. Single Responsibility

- Each function should do ONE thing
- Easy to name, test, and understand

2. Meaningful Names

- Use verbs: `calculateArea()`, `printMenu()`
- Descriptive: `findMaximum()` not `func1()`

3. Keep Functions Short

- Ideally fits on one screen
- If too long, break into smaller functions

4. Minimize Parameters

- Too many parameters = function doing too much
- 3-4 parameters maximum is good practice

5. Avoid Global Variables

- Use parameters and return values instead
- Prevents unexpected side effects

When to Create a Function

Create a function when:

- Code is repeated in multiple places
- A task can be clearly named
- Code block is becoming too long
- You need to test a specific operation
- Logic is complex and needs isolation

Don't create a function when:

- It's used only once in a simple context
- It's just one or two lines
- It doesn't have a clear purpose

Exercise Set 1: Basic Functions

Try these on your own:

1. Write a function to find the minimum of three numbers
2. Write a function to calculate the cube of a number
3. Write a function to check if a number is positive, negative, or zero (return 1, -1, or 0)
4. Write a function to convert Fahrenheit to Celsius:
$$C = (F - 32) \times 5/9$$
5. Write a function to find the average of three numbers
6. Write a function to print your name n times
7. Write a function to check if a year is a leap year
8. Write a function to find the larger of two numbers (similar to `max`)

Exercise Set 2: Intermediate Functions

More challenging problems:

1. Write a function to reverse the digits of a number
(e.g., $1234 \rightarrow 4321$)
2. Write a function to count digits in a number
3. Write a function to check if a number is palindrome
(same forwards and backwards: 121, 1331)
4. Write a function to find GCD of two numbers
5. Write a function to generate Fibonacci numbers up to n
6. Write a function to print multiplication table of a number
7. Write a function to calculate simple interest:
 $SI = (P \times R \times T)/100$

Exercise Set 3: Recursive Functions

Practice recursion:

1. Write recursive function to calculate sum from 1 to n
2. Write recursive function to find factorial
3. Write recursive function to calculate x^n
4. Write recursive function to find nth Fibonacci number
5. Write recursive function to print numbers from n to 1
6. Write recursive function to find sum of digits
7. Write recursive function to reverse a number
8. Write recursive function to find GCD (using Euclidean algorithm)

Hint: Always identify base case first!

What We Learned Today

1. **Functions** break programs into manageable pieces
2. **Function prototype** tells compiler about function
3. **Function definition** contains actual code
4. **Function call** executes the function
5. **Parameters** pass data to functions
6. **Return values** send data back
7. **Pass by value** creates copies of arguments
8. **Local variables** exist only in their function
9. **Random numbers** need `rand()` and `srand()`
10. **Recursion** is function calling itself

Function Syntax Quick Reference

Complete function template:

```
return_type function_name(parameter_list) {  
    // local variable declarations  
    // statements  
    return value;  
}
```

Three steps to use a function:

1. Declare (prototype): `int add(int, int);`
2. Define: Write the actual code
3. Call: `result = add(5, 3);`

Important Points to Remember

- Always include function prototype before `main`
- Prototype ends with semicolon, definition doesn't
- Return type must match what function returns
- Number and type of arguments must match parameters
- Changes to parameters don't affect original variables
- Use `void` for no parameters or no return value
- Call `srand(time(NULL))` once for random numbers
- Recursive functions need a base case
- Keep functions short and focused
- Use meaningful function names

Common Headers for Functions

Header	Purpose
<code><stdio.h></code>	Input/output: <code>printf</code> , <code>scanf</code>
<code><stdlib.h></code>	Utilities: <code>rand</code> , <code>srand</code>
<code><math.h></code>	Math functions: <code>sqrt</code> , <code>pow</code>
<code><time.h></code>	Time functions: <code>time</code>
<code><string.h></code>	String functions (Week 8)
<code><ctype.h></code>	Character functions (Week 8)

Remember: Include headers at the top of your program!

Tips for Success

1. Start Simple

- Write void functions first (no return value)
- Add parameters one at a time

2. Test Functions Separately

- Test each function individually
- Use simple test cases first

3. Use Comments

- Explain what function does
- Describe parameters and return value

4. Follow Naming Conventions

- Use lowercase with underscores: `calculate_area`
- Or camelCase: `calculateArea`
- Be consistent!

5. Practice, Practice, Practice!

- Write many small functions
- Rewrite old programs using functions

Next Week: Arrays

Coming up in Week 6:

- What are arrays?
- Storing multiple values of same type
- Accessing array elements
- Passing arrays to functions
- Multi-dimensional arrays
- Common array operations

Why arrays are important:

- Store grades for 100 students
- Process lists of numbers
- Work with strings (array of characters)
- Build more complex programs

Homework Assignment

Complete these before next class:

1. Write a program with function to check if number is Armstrong number (e.g., $153 = 1^3 + 5^3 + 3^3$)
2. Create a menu-driven calculator with functions for:
 - Addition, Subtraction, Multiplication, Division
3. Write a function to generate first n prime numbers
4. Create a program to simulate coin toss 100 times and count heads/tails
5. Write recursive function to calculate sum of array elements (hint for next week!)

Read: Chapter 6 on Arrays before next class

Questions?

*"Functions are the building blocks of programs.
Master them, and you master programming!"*

Thank You!

See you next week with Arrays!

Keep practicing and happy coding!

Bonus Example 1: Swap Function (Won't Work!)

This will NOT swap the values:

```
1 void swap(int a, int b) {  
2     int temp = a;  
3     a = b;  
4     b = temp;  
5     // Only swaps local copies!  
6 }  
7  
8 int main(void) {  
9     int x = 5, y = 10;  
10    swap(x, y);  
11    printf("x=%d, y=%d\n", x, y); // Still x=5, y=10  
12    return 0;  
13 }
```

Why? Pass by value! We'll learn how to fix this with pointers in Week 9.

Bonus Example 2: Function Returning Multiple Results

Problem: Functions can only return one value

Workaround: Return a value, print others

```
1 int calculate(int a, int b) {  
2     printf("Sum: %d\n", a + b);  
3     printf("Difference: %d\n", a - b);  
4     return a * b;    // Return product  
5 }  
6  
7 int main(void) {  
8     int product = calculate(10, 5);  
9     printf("Product: %d\n", product);  
10    return 0;  
11 }
```

Better solution: Use pointers (Week 9) or structures (Week 10)

Bonus Example 3: Nested Function Calls

```
1 #include <stdio.h>
2
3 int square(int x) { return x * x; }
4 int add(int a, int b) { return a + b; }
5
6 int main(void) {
7     // Calculate (3^2) + (4^2)
8     int result = add(square(3), square(4));
9     printf("Result: %d\n", result); // 9 + 16 = 25
10
11     // Can also nest deeper
12     int x = square(add(2, 3)); // square(5) = 25
13     printf("x: %d\n", x);
14
15     return 0;
16 }
```

Note: Inner functions execute first!